

02-Introduction to Git GitHub and Version Control

Introduction

- The session serves as an introductory lecture and hands-on tutorial on using Git, GitHub, and core version control concepts for collaborative software development.

Installation and Setup

- Participants are guided to ensure Anaconda (for Python/Jupyter) and Git are both installed.
- Detailed instructions for downloading and installing Git on Windows (using the standalone installer).
- Steps to create a GitHub account, with solutions for common sign-up and email issues.
- Discussion on proper linking between local Git and GitHub accounts (matching user name/emails).
- Troubleshooting installation obstacles, including file path issues, admin/user mismatches, and terminal/command prompt distinctions.
- Linux/Mac users advised on direct terminal usage with equivalent commands.

Core Concepts

What is Git?

- Git is defined as an open-source version control system, mainly for tracking and collaborating on source code.

- Tracks changes in code, supports collaboration, nonlinear development, and rollbacks.
- Integral for team projects, allowing individual contributions and maintaining code history.

Version Control Systems

- Centralized VCS: Uses a single server; all files/versions stored centrally—highly vulnerable to server failure.
- Distributed VCS: Each user/system has a full version of the code; more robust against failures, supports offline work, needs periodic synchronization (pull/push between local and server).

Key Terms

- Source Code: The actual code files for a project (e.g., .py for Python).
- Repository: Central location for storing and tracking versions of code/files (locally and on GitHub).
- Branching: Enables parallel development on features without disturbing the main code; merged when stable.

Git Workflow

- Three-stage process: Working Directory → Staging Area → Local Git Repository.
- Typical flow:
 - Create or modify file(s) in working directory.
 - Stage changes (prepare snapshot).

- Commit to the repository (record changes).
- Allows for rollback and recordkeeping of every change, who made it, and when.
- Explains reverting to previous versions (important for error corrections and testing new features).

GitHub Overview

- GitHub is an online hosting service for Git repositories, enabling collaborative, distributed workflows.
- Features public/private repositories, project sharing, version tracking, and issue management.
- Anyone can view/fork public code, contribute, and propose improvements.
- Other platforms like Bitbucket are mentioned as alternatives.
- Facilitates teamwork, transparent code history, and enhanced code safety and reusability.

Hands-On Practice and Command-Line Usage

Basic Commands Demonstrated

- `git config --global user.name <name>` and `git config --global user.email <email>`
: Set up user identity.
- Navigation: `pwd`, `ls`, `cd`, `mkdir`, etc., for file system management, finding/creating directories.
- `git clone <url>`: Download (clone) a remote GitHub repository to a local system.

- Using Vim (`vim <file>`), `cat <file>` to create, edit, and view files.
- Explanation of hidden `.git` folder marking a directory as a repository.
- `git status` to check repository status and confirm setup.
- Examples provided for both Windows and Linux environments.

Repository Management

- How to create a new GitHub repository, set it public, and add README for initial content.
- Cloning the repository into a desired folder (stepwise file navigation suggestions).
- Workflow ensures project organization and proper Git integration.

Troubleshooting and Q&A

- Instructor addresses common errors: invalid email/user, command syntax, terminal issues, file/path mismatches.
- Guidance on dealing with command-line errors (e.g., command not found, missing directories).
- Tips: check folder structure with `ls`, verify `.git` existence, proper use of commands.
- Suggestions to reinstall Git if persistent problems occur; cross-reference video guides for exact steps.
- Differences in workflow for various OS (Windows, Linux, Mac) pointed out where relevant.
- Encouragement to follow each step attentively, using correct paths and names.

Closing Notes

- Class ends with a brief preview of pushing code to GitHub (to be covered next session).
- Encouragement to review the session recording and repeat steps for practice.
- Troubleshooting sessions promised for unresolved individual errors.
- Reminder: Git and GitHub skills are essential for coding, collaboration, and are foundational for upcoming Python content.

Notes powered by [CraftNote](#)